

A HANDS-ON FIELD GUIDE

Ship It with Claude Code

Build one real project from an empty folder to a live website, with every prompt shown.



```
> build a storefront for my PDF shop,  
test it, then push it to GitHub
```

```
✓ index.html created  
✓ pushed to origin/main
```

Before you start

Most AI coding guides show you a list of features. This one shows you a **build**. We start with an empty folder and finish with a real, live website that takes payments. It's the same storefront you may have bought this PDF from.

Along the way you will see the actual prompts, the plans Claude Code proposed, the bugs it made, and how we fixed them. You get the messy middle, not a highlight reel.

Who this is for

- **Students and beginners** who can read a little HTML or Python and want to build real things faster.
- **Solo builders** with an idea (a shop, a portfolio, a tool) and no team.
- **Developers** who've tried AI chat tools and want a steadier, more reliable workflow.

What you'll be able to do

- ☐ Set up Claude Code and give it a proper project brief
- ☐ Plan a build before any code is written
- ☐ Review, test and fix what the AI writes, instead of trusting it blindly
- ☐ Put a site on GitHub and online for free, with a working Buy button
- ☐ Save your best workflows as reusable commands

What you need

A computer (Windows, macOS or Linux), a free GitHub account, and a Claude plan or Anthropic API account that includes Claude Code. No paid hosting is needed.

HOW TO READ THIS GUIDE

Parts 1 and 2 are meant to be followed in order, at your keyboard. Part 3 and the appendices are for dipping into later. The prompts are in shaded boxes. Copy them, then change the details to fit your own project.

Contents

PART 1 • SET UP

1	How Claude Code actually works	5
2	Install it and run your first session	7
3	CLAUDE.md: the brief that changes everything	10

PART 2 • THE BUILD

4	Plan before you code	14
5	Build the first page	16
6	See it, break it, fix it	18
7	Git and GitHub without the fear	20
8	Go live for free	22
9	Add a real Buy button	24

PART 3 • LEVEL UP

10	Ten prompt patterns that work	27
11	Managing context: the skill nobody teaches	29
12	Automate yourself: commands, hooks, skills, agents	31
13	When things go wrong	33

APPENDICES

A	One-page cheat sheet	35
B	CLAUDE.md starter template	36
C	Prompt library	37
D	Ship-day checklist	39

© 2026 Mahdi Hasan Shadi / PageNook. For personal use; please don't redistribute.
Written and tested with the help of Claude Code. Tools change quickly: if a command in this guide behaves differently for you, run `/help` inside Claude Code or check the official docs at docs.anthropic.com. Claude and Claude Code are products of Anthropic; this guide is independent and not affiliated with or endorsed by Anthropic.

01

How Claude Code actually works

It isn't a chatbot that writes code. It's closer to a junior developer with access to your terminal.

When you use an AI chat window for coding, you are the one doing the work: you copy code out, paste it into files, run it, copy the error back in, and repeat. **Claude Code removes the copy-paste.** It runs inside your project folder and can do the steps itself:

- **Read** your files, and search across the whole project.
- **Write and edit** files, showing you each change as a diff.
- **Run commands** in your terminal, such as tests, builds, git and installs.
- **Look at the results** of those commands and decide what to do next.

That last point is the big one. Claude Code works in a loop: it acts, checks what happened, and adjusts. If a test fails, it reads the failure and tries a fix without waiting for you to paste anything.

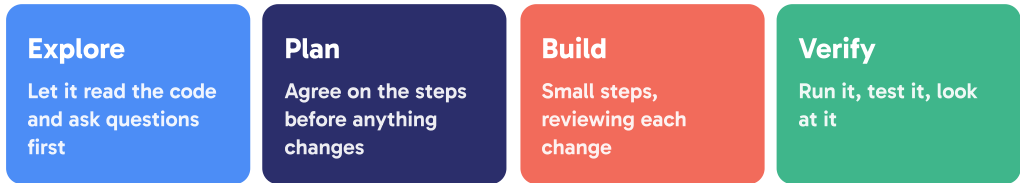
You are the director

Think of a film director and a talented crew. The crew does the hands-on work, but the director decides what the film is about, approves each scene, and says "cut" when something is going wrong. That's your job here:

- **You decide what to build** and what "done" looks like.
- **You approve risky actions.** By default, Claude Code asks before editing files or running commands.
- **You review the result.** It's fast and usually right, but it can be confidently wrong.

The four-step loop

Every successful session in this guide follows the same rhythm. Learn it once and use it everywhere.



Most bad results come from skipping the first two steps. "Build me a website" produces a generic website. "Read my notes, ask me three questions, then propose a plan" produces *your* website.

THE HONEST LIMITS

- It can misunderstand vague requests and build the wrong thing very efficiently.
- It can claim something works without actually checking. Always ask it to verify.
- In long sessions it can lose track of early details. Chapter 11 covers how to prevent that.
- It doesn't know your business, your taste or your deadlines unless you tell it.

10

Ten prompt patterns that work

Copy these shapes. Swap in your own details.

1 • Ask me first

Add *"Before you start, ask me up to 3 questions."* This brings hidden assumptions to the surface before they turn into code.

2 • Cause before cure

"Don't change anything. Explain why this happens." Use it for every bug that isn't obvious.

3 • Show, then save

"Show me the diff before saving." Use it for anything touching config, money or data.

4 • Point, don't describe

"Match the card style in @index.html." A real example beats a paragraph of description.

5 • Define done

"Done means: works at 375px, no console errors, all links jump to the right section." Then it can check itself against that.

6 • Prove it

"Run it and show me the output." Don't accept "this should work". Ask for evidence.

7 • Options, then choose

"Give me 3 different approaches with trade-offs. Don't implement." This is useful for design and architecture choices.

8 • Critic mode

"Review this like a strict senior engineer. What would you reject?" Asking it to critique its own work catches a surprising amount.

9 • Explain like I'm learning

"Explain what this function does in plain words, line by line." Never ship code you couldn't explain to someone else.

10 • Fence it in

"Only change the footer. Don't touch any other section." Clear limits stop it from "improving" things you didn't ask about.

WEAK PROMPT

add a contact page

STRONG PROMPT

Add contact.html matching the style of @index.html. It shows my email as text with a copy button, and a short "we reply within 2 days" line. Link it from the footer. Only touch these two files. Check it at 375px.



One-page cheat sheet

YOU WANT TO...	DO THIS
Start in a project	<code>cd project</code> then <code>claude</code>
Create the project brief	<code>/init</code> , then edit CLAUDE.md
Plan without changing anything	Shift+Tab to plan mode
Stop it right now	<code>Esc</code>
Rewind to an earlier point	<code>Esc</code> <code>Esc</code>
Point at a file	<code>@path/to/file</code>
Run a shell command yourself	<code>!git status</code>
Start a fresh task	<code>/clear</code>
Shrink a long session	<code>/compact</code>
Reopen the last session	<code>claude --continue</code>
Pick an older session	<code>claude --resume</code>
See all commands	<code>/help</code> or type <code>/</code>
Save a routine as a command	<code>.claude/commands/name.md</code>
Connect another tool	<code>claude mcp add ...</code>
Run one prompt from a script	<code>claude -p "..."</code>

The loop

Explore → Plan → Build → Verify. Commit between each working step.